Assignment No 12

Name: Sciddhanto Sinha
Class: CSE, SY-2(B)
Roll No.:2213111
Enrollment No.: MITU21BTCS0553

Aim:

Write a generic program to handle exception generated in following scenarios:

- Division by Zero
- Accessing a file which does not exist.
- Addition of two incompatible types
- Trying to access a nonexistent index of a sequence

Theory:

Error in Python can be of two types i.e. Syntax errors and Exceptions. Errors are the problems in a program due to which the program will stop the execution. On the other hand, exceptions are raised when some internal events occur which changes the normal flow of the program.

Difference between Syntax Error and Exceptions

Syntax Error: As the name suggests this error is caused by the wrong syntax in the code. It leads to the termination of the program.

Example:

```
# initialize the amount variable

amount = 10000


# check that You are eligible to

# purchase Dsa Self Paced or not

if(amount > 2999)

print("You are eligible to purchase Dsa Self Paced")
```

Exceptions: Exceptions are raised when the program is syntactically correct, but the code resulted in an error. This error does not stop the execution of the program, however, it changes the normal flow of the program.

Example:

```
# initialize the amount variable
marks = 10000

# perform division with 0
a = marks / 0
print(a)
```

In the above example raised the ZeroDivisionError as we are trying to divide a number by 0.

Try and Except Statement – Catching Exceptions

Try and except statements are used to catch and handle exceptions in Python. Statements that can raise exceptions are kept inside the try clause and the statements that handle the exception are written inside except clause.

Example: Let us try to access the array element whose index is out of bound and handle the corresponding exception.

```
# Python program to handle simple runtime error
#Python 3

a = [1, 2, 3]
try:
    print ("Second element = %d" %(a[1]))

    # Throws error since there are only 3 elements in array
    print ("Fourth element = %d" %(a[3]))

except:
    print ("An error occurred")
```

Output

Second element = 2

An error occurred

In the above example, the statements that can cause the error are placed inside the try statement (second print statement in our case). The second print statement tries to access the fourth element of the list which is not there and this throws an exception. This exception is then caught by the except statement.

Catching Specific Exception

A try statement can have more than one except clause, to specify handlers for different exceptions. Please note that at most one handler will be executed. For example, we can add IndexError in the above code. The general syntax for adding specific exceptions are –

```
try:
    # statement(s)
except IndexError:
    # statement(s)
except ValueError:
    # statement(s)
```

Example: Catching specific exception in Python

```
# Program to handle multiple errors with one
# except statement
# Python 3

def fun(a):
    if a < 4:

        # throws ZeroDivisionError for a = 3
        b = a/(a-3)

    # throws NameError if a >= 4
    print("Value of b = ", b)
```

```
try:
    fun(3)
    fun(5)

# note that braces () are necessary here for
# multiple exceptions
except ZeroDivisionError:
    print("ZeroDivisionError Occurred and Handled")
except NameError:
    print("NameError Occurred and Handled")
```

Output

ZeroDivisionError Occurred and Handled

Finally Keyword in Python

Python provides a keyword finally, which is always executed after the try and except blocks. The final block always executes after normal termination of try block or after try block terminates due to some exception.

Syntax:

```
try:
    # Some Code....

except:
    # optional block
    # Handling of exception (if required)

else:
    # execute if no exception

finally:
    # Some code.....(always executed)
```

Example:

```
# Python program to demonstrate finally

# No exception Exception raised in try block
try:
    k = 5//0 # raises divide by zero exception.
    print(k)

# handles zerodivision exception
except ZeroDivisionError:
    print("Can't divide by zero")

finally:
    # this block is always executed
    # regardless of exception generation.
    print('This is always executed')
```

Output:

```
Can't divide by zero
This is always executed
```

Program:

```
#trying division by zero
try:
    num1 = int(input("Enter First Number: "))
    num2 = int(input("Enter Second Number: "))

    result = num1 / num2

    print(result)
except ValueError as e:
    print("Invalid Input Please Input Integer...")
except ZeroDivisionError as e:
    print(e)
finally:
    print("Division by zero exception handled successfully")
#addition of two incompatible type
try:
    num1 = int(input("Enter First Number: "))
    num2 = int(input("Enter Second Number: "))

    result = num1 + num2

    print(result)
except ValueError as e:
    print("Invalid Input Please Input Integer...")

finally:
    print("Addition of incompatible types exception handled successfully")
#accessing non existent index of a sequence
```

```
a = [1, 2, 3]
try:
    print ("Second element = %d" %(a[1]))

    # Throws error since there are only 3 elements in array
    print ("Fourth element = %d" %(a[3]))

except IndexError as e:
    print (e)
finally:
    print("Accessing non existent index of a sequence exception handled successfully")



#opening a non existing file
# trying to open a file, which does not exist
try:
    #trying to open a file in read mode
    fo = open("myfile.txt","rt")
    print("File opened")
except FileNotFoundError:
    print("File does not exist")
except:
    print("Other error")
finally:
    print("Opening non existent file exception handled successfully")
```

Output:

```
Enter First Number: 3
```

Enter Second Number: 0

division by zero

Division by zero exception handled successfully

Enter First Number: 2

Enter Second Number: 3.3

Invalid Input Please Input Integer...

Addition of incompatible types exception handled successfully

Second element = 2

list index out of range

Accessing non existent index of a sequence exception handled successfully

File does not exist

Opening non existent file exception handled successfully

Conclusion: We have successfully learnt the concept of exception handling in Python and handled various exceptions like:

- Division by Zero
- Accessing a file which does not exist.
- Addition of two incompatible types
- Trying to access a nonexistent index of a sequence